Intent-Compiled Verified Execution for Reliable Stateful Tool Use

Anonymous ACL submission

Abstract

Language agents increasingly operate state-mutating tools, where one wrong argument or repeated action can cause invalid calls, unsafe state changes, and costly repair loops. We study **Intent-Compiled Verified Execution (ICVE)**, a runtime-checked framework for covered high-risk intents. ICVE compiles a request into a typed intent frame and delegates evidence acquisition, slot grounding, precondition repair, abstention, execution, and postcondition checking to deterministic state machines. The model still identifies intent, but no longer owns mutating control flow for registered task families. On public ToolSandbox single-turn and insufficient-information tasks, ICVE changes the failure mode: covered mutating requests become checked transitions or guarded abstentions with zero invalid or unsafe calls, while using far fewer model calls and tokens than ReAct. The pattern holds across local Qwen2.5 scales, a Phi-3-mini check, and hosted DeepSeek replications. AppWorld evaluations show the same coverage-risk profile on the official dev split and local `test_normal.txt`, while uncovered `test_challenge` families are reported as guarded no-action outcomes rather than forced mutations.

1 Introduction

Stateful tool use differs from stateless API answering. A single wrong argument can delete the wrong contact, send a message under the wrong precondition, or repeat an already completed state transition until the execution environment raises an error. Existing tool-use agents often represent this entire process as a text loop: the language model chooses a tool, fills arguments, interprets observations, repairs errors, and decides when to stop. ReAct-style agents, tool-learning systems, and recent interactive benchmarks make this setting increasingly realistic (Yao et al., 2022; Schick et al., 2023; Lu et al., 2024; Trivedi et al., 2024; Wang et al., 2025), but they leave open where stateful execution should be verified.

Our thesis is that many failures are not reasoning failures alone. They are *execution-boundary* failures: the model is asked to perform intent compilation, state tracking, evidence lookup, precondition management, mutation, and completion detection through plain text. Larger models help, but they do not remove invalid calls or unsafe state changes in insufficient-information settings.

ICVE changes that boundary. The LLM may still handle uncovered or low-risk requests, but registered state-sensitive intents are routed through a runtime compiler that emits runtime-checked tool traces. The reusable unit is not a prompt template but a declared intent machine whose runtime owns slots, evidence, repair, abstention, transitions, and postcondition stopping. The headline evaluation uses manually declared machines; the artifact also includes a restricted affordance-induction prototype for regular boolean setting APIs. The claim is an explicit, auditable boundary for covered task families and signature-identifiable affordances, not general machine synthesis or universal tool safety.

The positioning is deliberately narrow. ICVE is not merely stricter schema validation: a function schema can reject an ill-typed call, but it does not decide which evidence must be acquired before a mutation, whether an earlier transition already satisfied the goal, or whether missing state should force abstention. Nor is ICVE only semantic parsing from utterances to API calls: the compiled object is an executable state machine whose transition is checked against a live ledger. Unlike proof-carrying prompting, recurring proof obligations are encoded in the registered machine rather than regenerated by the model. Throughout the paper, "verified" means runtime checking against declared machine invariants, not a formal proof of

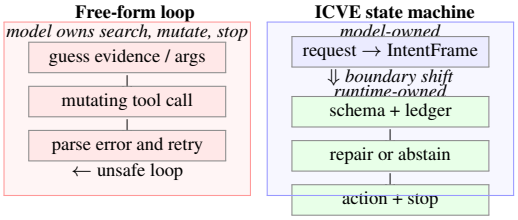

Figure 1: The execution-boundary shift. A free-form agent owns evidence search, mutation, error recovery, and stopping, so missing state can become invalid calls or unsafe loops. ICVE restricts the model to typed frame selection and moves ledger checks, repair, abstention, tool execution, and postcondition stopping into the runtime.

arbitrary agent behavior.

This paper makes three contributions:

1. A typed intent/state-machine abstraction for stateful tool use. Each risky task family is represented as an `IntentMachine` with an explicit schema, compiler, and handler.

2. A runtime-checked executor that owns evidence acquisition, argument grounding, precondition repair, insufficient-information abstention, and completion detection.

3. A public-benchmark evaluation on ToolSandbox and AppWorld showing a failure-mode shift from invalid or unsafe mutation toward checked transitions and guarded abstention on covered families, plus a bounded no-LLM diagnostic showing that regular boolean-setting affordances can be induced from previously unseen API signatures.

2 Method

2.1 Typed Intent Machines

ICVE represents each covered stateful task family as an intent machine:

```
Machine  = (schema, compiler, handler)
schema   = (intent_type, slots, terminal_states)
compiler = request x dialogue x tools -> frame
    or None
handler  = frame x ledger x tools -> tool_action
     or final_action
```

An `IntentFrame` is the runtime-owned state for a user intent:

```
IntentFrame = (
  intent_type,
  slots: name -> (value, required, source),
  state,
  missing_slots,
  abstain_reason
)
```

The compiler is deliberately typed and conservative. It does not ask the model to emit a proof or a full tool trace. Instead, it maps a user request into a small frame whose slot values are extracted from the request, copied from prior observations, or produced by a canonicalizer. In the ToolSandbox binding, the registry currently covers 13 intent families: currency conversion, stock lookup, reverse geocode, current city, last-message contact update, bulk contact relationship update, contact name lookup, location phone lookup, holiday timestamp/countdown, weather, distance, and reminder.

The machine interface is also the method's trust boundary. A model may choose the wrong intent type or slot; ICVE mitigates that risk with schema validation, conservative compilers, domain verifiers, and abstention. For covered intents, the model cannot choose arbitrary state-changing calls: handlers emit only actions derived from validated slots and ledger observations, and a policy verifier can reject or rewrite the action. Thus the claim is conditional and inspectable: behavior depends on the registered machine and verifier, not on an opaque promise that the model will reason safely.

2.2 Compilation

Compilation proceeds by registry order. Figure 1 illustrates the resulting boundary: the compiler selects a frame, while the runtime owns state checks, repair, abstention, execution, and stopping. On each turn, ICVE first runs an insufficient-information verifier. If required tools or information are unavailable, the verifier returns an abstention frame. Otherwise, the runtime tries registered compilers until one returns a frame, and validates that frame against its schema. If no registered machine applies, control falls back only for uncovered low-risk requests.

This loop is implemented in a benchmark-independent runtime named `RaveRuntime` in the artifact for historical reasons. Domain bindings provide intent machines, a state-ledger interface, and a verifier for final-message wording and transition rejection. The generic runtime owns registry lookup, uniqueness checks, frame validation, completion/abstention hooks, and handler dispatch. A policy verifier can rewrite a `ToolAction` into a

FinalAction, giving each domain binding one explicit boundary for unsafe, unsupported, or previously failed transitions.

The artifact also contains an intentionally narrow affordance-template induction path. When enabled, the runtime records unsupported requests, discovers regular boolean setting affordances from tool names/signatures of the form get_*_status() and set_*_status(on: bool), proposes a candidate IntentMachine, executes it in shadow mode, and promotes it only if tool-signature invariants, emitted-action whitelists, and unrelated-request counterexamples pass. This dynamic path is a coverage diagnostic for signature-identifiable stateful affordances, not evidence that arbitrary state machines can be synthesized.

2.3 Runtime-Checked State Machines

The runtime consumes a state ledger rather than trusting raw dialogue text. The ledger records user-provided values, successful tool observations, and recent tool errors; a ToolSandbox ledger instantiates this interface with settings, contacts, messages, and reminders. A state-machine handler consumes the current IntentFrame and ledger, then emits at most one transition:

```
ToolAction  = (tool, args, reason)
FinalAction = (message, reason)
```

For a registered machine, a mutating ToolAction is admissible only when the frame is schema-valid, required slots are grounded, preconditions hold or a bounded repair is emitted, the postcondition is not already satisfied, and the action belongs to the handler's transition set. Otherwise the runtime must stop, abstain, or route to a low-risk fallback. This is the measured runtime invariant, not a proof about arbitrary unregistered behavior.

For read-only actions, the verifier normalizes arguments and blocks empty or ungrounded searches. For state-changing actions, it checks schema conformance, argument grounding, state preconditions, and duplicate completion before allowing exactly one transition. If the requested postcondition is already satisfied, the runtime stops. When a safe repair exists, such as disabling low-battery mode before enabling Wi-Fi, the runtime emits the repair action before the requested mutation. When repair would require missing information or unavailable tools, it abstains instead of letting the model guess. Thus the verifier is stricter than a call schema: it checks the transition's evidence, preconditions, and completion state, not only the argument types of one invocation.

The design point is that stateful execution is runtime-owned: registered intents are compiled and executed through deterministic machines whose assumptions are visible in code, while uncovered or low-risk requests can still use the model directly.

3 Experimental Setup

We evaluate under real local LLM inference rather than oracle policy traces. The main agent LLM is served through a local OpenAI-compatible CUDA server. ToolSandbox tools that would otherwise require external RapidAPI calls use deterministic offline fixtures so the benchmark remains repeatable.

All reported local-model experiments use temperature 0 inference on a single NVIDIA RTX 3080 Ti GPU with 12GB memory. Qwen2.5-7B and Phi-3-mini are run with 4-bit quantization; Qwen2.5-0.5B and Qwen2.5-3B are run as local instruction models through the same OpenAI-compatible interface. No model training or fine-tuning is performed.

For the hosted replication, we run the same ToolSandbox suites with the DeepSeek API model identifiers deepseek-chat and deepseek-reasoner through the same OpenAI-compatible client. API keys are loaded from environment variables and are not written to command lines or result files.

For AppWorld, we use two settings. First, we run an isolated pctu-appworld environment on a targeted public stateful slice covering 72 tasks across twenty-four registered AppWorld intent machines; some hosted DeepSeek rows are combined from an earlier 66-task run plus the final six-task Spotify-family expansion. Second, we run ICVE on the complete local official AppWorld 0.2.0 dev.txt split in a separate pctu-appworld-agents environment and evaluate with AppWorld's packaged evaluator. The deterministic path uses typed compilers only; the real-LLM paths use local Qwen2.5-3B, DeepSeek-chat, or DeepSeek-reasoner to produce a typed intent frame and then execute that frame through the same runtime-checked executor. AppWorld handlers interact only through the live apis object;

the runtime policy blocks code that references task files, ground truth, compiled solutions, or oracle data.

Metrics are task success, ToolSandbox similarity, invalid tool calls per task, unsafe state changes per task, verifier rejections, runtime repair actions, model calls, and a token proxy computed from local model usage. The kill criterion is not only safety: a candidate must reduce invalid calls and unsafe state changes without letting repair cost erase task-success or token-efficiency gains. For AppWorld, we report evaluator success, unsafe state-test failures, failed API repair attempts, API calls, model calls, and model-reported token usage.

We use Wilson 95% intervals over the fixed episode counts in each main suite; the complete interval table is archived in the supplement. Invalid and unsafe counts are reported as per-episode means on the same fixed slices.

The experiments answer three questions: whether ICVE reduces invalid or unsafe state changes, whether the same boundary transfers across models and environments, and whether typed machines plus abstention are necessary beyond intent recognition. The anonymous supplement and 4open.science mirror include source code, task-id lists, evaluator summaries, and commands for reproducing the reported rows, while excluding credentials, raw logs, databases, and downloaded bundles.

Machine construction protocol. Static machines are developed from API signatures, public task instructions, and ordinary runtime error categories, not from ground-truth answers, private database state, compiled solutions, or successful trajectories. The AppWorld runner enforces this discipline at execution time by loading only the public instruction and routing all state access through the live `apis` object. Machines are registered at the intent-family level: a compiler emits reusable slots and a handler must either operate through public APIs or return an unsupported no-action outcome. We audit this boundary with per-machine coverage/LOC tables, the full local `test_normal.txt` coverage diagnostic, and a static public-instruction compile audit over `test_normal.txt` and `test_challenge.txt`.

Baselines. We compare against ReAct (Yao et al., 2022), a plain JSON tool-use loop with no runtime intent compiler; ReAct+guardrail (PCTU), a stronger proof-carrying baseline that keeps the model in charge of the action loop but requires each mutating call to carry a schema, evidence, preconditions, and postconditions checked by a runtime verifier; and ICVE ablations that remove the typed DSL or the abstention verifier.

Implementation audit. We checked the large ICVE–ReAct gaps against three common experimental artifacts. First, the ToolSandbox rows use the same scenario resolver, available tools, model endpoint, temperature, message budget, and evaluator for all methods; the ReAct agent receives the same tool signatures as ICVE, and its tool calls are executed through the same environment-facing code path. Second, AppWorld comparisons are evaluated by AppWorld's packaged evaluator on the same task files where applicable; the official dev57 ReAct row uses the upstream `simplified_react_code_agent` with its default 50-step budget. Third, the AppWorld runner now loads only the public task instruction before execution, and both runtime and prompting policies block references to task files, ground-truth data, compiled solutions, or oracle fields. These checks do not make the comparison universal; they rule out known wrapper bugs, evaluator mismatch, and hidden answer access. The remaining threat is coverage: ICVE is strong where an accurate machine is registered, and abstains or falls back outside that coverage.

4 Results

4.1 MiniStore Diagnostic

MiniStore is a small local stateful environment used as a diagnostic rather than as the headline benchmark. It tests whether simply adding a schema/proof/verifier guardrail to ReAct is enough.

method	succ.	goal	invalid	unsafe	LLM	tokens
ReAct	0.375	1.000	1.125	1.125	5.500	2477.750
ReAct+guardrail	0.250	0.250	0.000	0.000	7.250	3169.375
ICVE	1.000	1.000	0.000	0.000	2.250	603.875

Table 1: MiniStore real-LLM diagnostic with Qwen2.5-3B. ReAct+guardrail (PCTU) suppresses invalid and unsafe actions, but still leaves the model to author proof-bearing actions and pays too much proof/repair overhead.

4.2 Public ToolSandbox

The strongest evidence is on ToolSandbox (Lu et al., 2024). Table 2 reports the primary Qwen2.5-

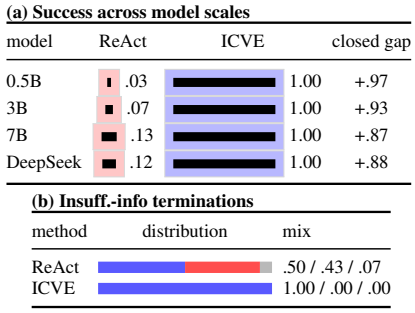

Figure 2: Deep ToolSandbox analysis. ICVE closes the success gap across weak, main, and hosted model scales and changes insufficient-information failures by eliminating unsafe state transitions. Blue denotes success, red unsafe mutation, and gray other invalid or incomplete termination.

3B results, and Figure 2 summarizes the scale and failure-mode effects.

The Wilson 95% confidence intervals for the primary success rates are separated from the mean-count columns: on the 73-episode single-turn suite, ICVE is 1.0000 $[0.9500, 1.0000]$ while ReAct is 0.0685 $[0.0296, 0.1505]$; on the 28-episode insufficient-information suite, ICVE is 1.0000 $[0.8794, 1.0000]$ while ReAct and the no-abstention ablation are both 0.5000 $[0.3263, 0.6737]$.

Table 3 extends the proof-carrying baseline beyond the MiniStore diagnostic to the full DeepSeek-chat insufficient-information suite: guardrails reduce but do not eliminate unsafe or invalid termination, whereas ICVE removes the mutating loop from the model for covered intents and executes checked transitions without agent LLM calls.

Dynamic-synthesis probe. To test whether ICVE can add a machine without pre-registering that family, we disable the static registry and enable the affordance-induction prototype described in Section 2.2. On five official ToolSandbox setting scenarios (`cellular_off`, `wifi_off`, and three low-battery repair cases), the runtime synthesizes one setting machine per task from the available API signatures, passes shadow-mode and counterexample checks, and reaches 5/5 success with zero LLM calls. We also run a held-out synthetic-affordance diagnostic with no static machines: five previously unseen APIs (`bluetooth`, `dark_mode`, `privacy_mode`, `roaming_data`, and `auto_sync`) are induced and mutated correctly, while an invalid `focus_mode` setter and an unrelated stock query are rejected. The diagnostic is intentionally limited to regular boolean setting APIs; it does not claim automatic synthesis for AppWorld purchase, Gmail, or vacation-expense machines.

4.3 Model Scale

The effect is not a Qwen2.5-3B-only artifact. Table 4 summarizes weak and stronger local model runs. The 7B result uses a local 4-bit quantized model and should not be interpreted as frontier-model coverage.

4.4 Cross-Family Diagnostic

We also ran a small non-Qwen diagnostic with Phi-3-mini: on 10 core ToolSandbox scenarios, ReAct reaches 0/10 success with 1.8 invalid calls per task, while ICVE reaches 10/10 with zero invalid/unsafe calls; on 10 insufficient-information scenarios, ReAct reaches 6/10 with 1.7 invalid and 0.3 unsafe calls per task, the no-abstention ablation keeps 6/10 and restores unsafe attempts, and ICVE reaches 10/10 with zero invalid/unsafe calls. This is not frontier-model evidence, but it addresses the narrow concern that the result is only a Qwen-family artifact.

4.5 Hosted OpenAI-Compatible Replication

We also replicate the primary ToolSandbox suites with the hosted DeepSeek-chat endpoint. Table 5 shows the same qualitative pattern: ICVE eliminates invalid and unsafe outcomes on both covered suites, while ReAct remains substantially less successful and more expensive. The no-DSL and no-abstention ablations again expose the two mechanisms: typed compilation is needed on single-turn stateful tasks, and abstention verification is needed when the user omits required information.

We also ran DeepSeek-reasoner on the same two suites; both full runs passed the kill criteria, and the recorded summaries are listed in the artifact manifest.

4.6 AppWorld

The same runtime-checked executor transfers to AppWorld on a targeted public state-changing slice. The active rows cover 72 tasks across twenty-four registered AppWorld intent machines spanning phone, Venmo, Gmail, Simple Note, Spotify, Amazon, alarms, Bucket List, and cross-app sharing tasks.

suite	method	n	succ.	invalid	unsafe	repair	LLM	tokens
single-turn	ReAct	73	0.0685	2.1507	0.0000	0.0000	11.8219	11048.0
single-turn	ICVE no DSL	73	0.4932	0.3288	0.0000	0.8082	3.9726	3307.9
single-turn	ICVE	73	1.0000	0.0000	0.0000	2.4658	0.1233	32.7
insufficient-info	ReAct	28	0.5000	2.1786	0.4286	0.0000	11.7500	9896.4
insufficient-info	ICVE no abstention	28	0.5000	1.5714	0.4286	0.3929	7.2857	6612.1
insufficient-info	ICVE	28	1.0000	0.0000	0.0000	1.5714	0.0000	0.0

Table 2: Main public ToolSandbox results with Qwen2.5-3B. Metrics after success are per-episode means. ICVE performs deterministic repair actions but still uses far fewer model calls and tokens; the typed DSL is necessary on single-turn tasks, and the abstention verifier is necessary on insufficient-information tasks.

method	succ.	invalid	unsafe	reject	LLM
ReAct	.321	1.214	.607	.000	7.500
ReAct+guardrail	.750	1.036	.179	2.214	6.786
ICVE	1.00	.00	.00	.00	.00

Table 3: DeepSeek-chat ToolSandbox guardrail diagnostic on the full 28-scenario insufficient-information suite. ReAct+guardrail (PCTU) improves safety by rejecting model-authored contracts, but leaves proof/action authoring in the model loop and retains nonzero invalid/unsafe calls and model cost.

The AppWorld result separates intent recognition from checked execution. Typed-intent code gives the model the same frame but still asks it to write the mutation code; it reaches 0/72 with Qwen2.5-3B and 7/72 with DeepSeek-chat. Direct code and multi-step code-observation variants likewise fail or become expensive, while DeepSeek repair reaches 21/72 but introduces unsafe changes. The error pattern is consistent: models often know the requested operation but fail at grounded execution. One pre-verifier DeepSeek-chat intent extraction produced `archive` for a delete instruction; the instruction-aware slot verifier rewrites the frame before execution. ICVE's residual repair cost is concentrated in Venmo transfers, where public metadata reveals card expiry but not hidden balances, so the runtime probes non-expired cards and then stops after success.

We then moved the ICVE binding to the complete local official AppWorld 0.2.0 `dev.txt` split. This adds the remaining official dev task families as registered intent machines, including navigation-based Spotify artist lookup, Venmo social-feed aggregation, file-prefix moves, current-artist follower lookup, and Simple Note markdown export. Table 7 reports AppWorld's packaged evaluator on all 57 dev tasks. Both deterministic ICVE and ICVE with DeepSeek-chat intent extraction complete all 57 local dev tasks with 100.0 task-goal / 100.0 scenario-goal completion, while the official DeepSeek ReAct-code agent reaches 79.0 / 73.7 and 45/57. The residual 0.0351 invalid API attempts per task are two Venmo payment-card probes: public card metadata exposes expiry but not balance, so the runtime tries one non-expired insufficient-balance card and then succeeds with the next public card. We do not use the private admin balance API to remove this residual repair cost.

The official DeepSeek ReAct-code dev57 run uses 879 LLM calls and 6.45M tokens. These are same-split local official-dev comparisons, not held-out test or leaderboard submissions.

As a held-out smoke beyond the official dev split, `test_normal` initially exposed zero coverage on the first 12 tasks; after registering four typed machines, ICVE reached 12/12. Expanding to the first 120 tasks gives 117/120 overall and 117/117 supported success, with the three then-unsupported trip-note debt-settlement tasks abstained. After adding one general machine for Simple Note trip-debt ledgers to private Venmo payments/requests, the full local file, including its final no-newline task ID, gives the deterministic profile 168/168 overall, 168/168 supported success, 0 unsafe changes, and 0 invalid calls. This is a coverage diagnostic, not a public leaderboard submission.

Coverage and development cost. The coverage assumption is auditable rather than implicit. ToolSandbox uses 13 static machines; AppWorld has 125 registered machines, of which 55 cover all 168 `test_normal.txt` tasks (3.05 tasks per used machine). For used AppWorld machines, the median slots/compiler LOC/handler LOC/total LOC are 1/18/74/94. A static public-instruction audit compiles 168/168 `test_normal` and 138/417 `test_challenge` instructions without tools, databases, or ground truth; the roadmap keeps unsupported Amazon purchase/search (155), Gmail (83), Splitwise (28), Spotify (12), and Venmo (1) buckets as cover-

model	suite	method	n	succ.	invalid	unsafe	tokens
0.5B	single-turn	ReAct	30	0.0333	5.6667	0.0000	8054.5
0.5B	single-turn	ICVE	30	1.0000	0.0000	0.0000	54.3
0.5B	insufficient-info	ReAct	28	0.6071	3.2500	0.3214	8367.3
0.5B	insufficient-info	ICVE	28	1.0000	0.0000	0.0000	0.0
7B-4bit	single-turn	ReAct	30	0.1333	0.4333	0.0000	3483.5
7B-4bit	single-turn	ICVE	30	1.0000	0.0000	0.0000	34.6
7B-4bit	insufficient-info	ReAct	28	0.4286	1.2143	0.3929	4894.4
7B-4bit	insufficient-info	ICVE	28	1.0000	0.0000	0.0000	0.0

Table 4: ICVE preserves the zero-invalid, zero-unsafe pattern across weak and stronger local Qwen2.5 scales; the primary Qwen2.5-3B results appear in Table 2.

suite	method	n	succ.	invalid	unsafe	tokens
single-turn	ReAct	73	0.1233	0.5342	0.0000	4397.1
single-turn	ICVE no DSL	73	0.5479	0.0548	0.0000	1258.6
single-turn	ICVE	73	1.0000	0.0000	0.0000	13.6
insufficient-info	ReAct	28	0.3214	1.2143	0.6071	7299.6
insufficient-info	ICVE no abstention	28	0.5000	0.6786	0.4286	3234.4
insufficient-info	ICVE	28	1.0000	0.0000	0.0000	0.0

Table 5: Hosted DeepSeek-chat ToolSandbox replication. Metrics after success are per-episode means; full outputs are archived in the artifact manifest.

age gaps. Forty held-out `test_challenge` slices now cover 120 additional tasks across phone, Gmail, file-system, payment-card, Venmo, Spotify, Amazon shopping, Amazon answer-only, subscription-history, and cross-app workflows, reaching 120/120 success with zero invalid or unsafe calls. These numbers are deliberately reported as machine-family reuse and coverage accounting, not as a full challenge-set score.

4.7 Multi-Turn Diagnostic

On a 28-episode hidden-task multi-turn diagnostic with Qwen2.5-0.5B, ICVE reaches 0.8929 success, 0.9773 mean similarity, 0 invalid calls, and 0 unsafe state changes. This is an extension result, not the headline claim: a small number of recency and benchmark-interface near-misses remain.

5 Analysis

The empirical pattern is that free-form agents overpay for tasks whose state semantics are compact but strict. ReAct repeatedly infers evidence needs, calls tools with fragile arguments, interprets observations, repairs failures, and detects completion. PCTU rejects bad calls, but still relies on model-authored proof-bearing actions. ICVE moves these recurring checks into the runtime. Runtime actions can increase, but model calls and token usage collapse because repeated model-mediated repair disappears.

The size of the improvement is therefore expected on covered tasks rather than evidence that the base model has learned substantially better reasoning. The covered families have reusable state semantics: ground an entity, establish read-side evidence, repair a small set of preconditions, mutate once or in a bounded loop, and stop when the postcondition holds. ReAct solves that control problem anew on every episode. Its failures concentrate in invalid JSON or unsupported arguments, missing evidence searches, premature or repeated mutations, and insufficient-information cases where a text loop guesses instead of abstaining. ICVE removes those degrees of freedom: the compiler chooses a typed frame, the handler emits only schema-valid actions over grounded slots, the ledger remembers observed state and failed calls, the verifier blocks unavailable or unsafe transitions, and the completion detector prevents duplicate actions. Table 8 summarizes this failure taxonomy.

Ablations support this mechanism. Removing the typed DSL leaves the model with runtime guards but loses much of the single-turn success and invalid-call control; removing abstention reproduces the unsafe insufficient-information pattern seen in ReAct. In AppWorld, giving the model a typed intent but still asking it to author mutation code does not close the gap, while routing the same frame through the runtime-checked executor does. The result is a boundary effect: ICVE is clean where an accurate machine covers the task, but this hand-engineered coverage does not imply broad success on uncovered families.

The added diagnostics make this boundary falsifiable. On ToolSandbox insufficient-information

method	n	succ.	inv./repair	unsafe	LLM	tokens
deterministic compiler	72	72/72	0.0694	0.0000	0.0000	180.5
Qwen2.5-3B intent	72	72/72	0.0694	0.0000	1.0000	3220.3
DeepSeek-chat intent	72	72/72	0.0694	0.0000	1.0000	3352.2
DeepSeek-reasoner intent	72	72/72	0.0694	0.0000	1.0000	3462.3
Qwen direct code	72	0/72	1.0000	0.0000	1.0000	1544.0
Qwen typed intent/code	72	0/72	1.0000	0.0000	2.0000	4674.8
Qwen code repair	72	0/72	2.7083	0.0000	2.8889	5342.8
Qwen multi-step code	72	0/72	3.7500	0.0000	4.3472	9031.7
DeepSeek direct code	72	4/72	0.8611	0.0139	1.0000	2028.9
DeepSeek typed intent/code	72	7/72	0.7917	0.0139	2.0000	5478.9
DeepSeek code repair	72	21/72	1.5556	0.0417	2.3472	5898.8
DeepSeek multi-step code	72	20/72	1.2222	0.0000	4.3056	12128.6

Table 6: Targeted public AppWorld slice. Metrics after success are per-task means. All rows use the same 72-task dataset; hosted DeepSeek rows are combined from incrementally flushed shards where needed. AppWorld's packaged evaluator confirms the corresponding task-completion counts: 72/72 for ICVE runtime rows, 0/72 for local Qwen code baselines, 4/72 for DeepSeek direct code, 7/72 for DeepSeek typed intent/code, 21/72 for DeepSeek repair, and 20/72 for DeepSeek multi-step code.

method	n	task	scen.	invalid	unsafe	LLM
deterministic compiler	57	100.0	100.0	0.0351	0.0000	0.0000
DeepSeek-chat intent	57	100.0	100.0	0.0351	0.0000	1.0000
official DeepSeek ReAct-code	57	79.0	73.7	–	–	15.4211

Table 7: Complete local official AppWorld 0.2.0 dev split for ICVE rows. Task and scenario columns are packaged-evaluator percentages; invalid, unsafe, and LLM are per-task means from the local runtime logs for ICVE and official runner logs for ReAct-code. The official ReAct-code row uses 6.45M tokens total.

tasks, ReAct produces 0.4286 unsafe mutations and 2.1786 invalid calls per task; ICVE reduces both to zero, while removing abstention restores the unsafe rate. On AppWorld, the 168-task local `test_normal.txt` diagnostic, 138/417 static `test_challenge` compile audit, and 120-task held-out slice collection expose both coverage and non-coverage. The dynamic diagnostic also induces five boolean-setting machines from unseen API signatures without model calls.

6 Related Work

ReAct-style agents interleave reasoning and action through text (Yao et al., 2022); Reflexion adds verbal self-feedback (Shinn et al., 2023), and Toolformer explores tool-use learning (Schick et al., 2023). Tool-learning systems and function-calling datasets scale API selection and invocation (Li et al., 2023; Shen et al., 2024; Qin et al., 2024; Patil et al., 2024; Liu et al., 2025; Dang et al., 2025), while ToolSandbox, AppWorld, BFCL, tau-bench, and agent benchmarks evaluate stateful and interactive use (Lu et al., 2024; Trivedi et al., 2024; BFCL; Yao et al., 2024; Liu et al., 2024; Zhou et al., 2024; Xie et al., 2024; Wang et al., 2025). Guardrails, schemas, proof-carrying prompts, and safety evaluations constrain calls (Ruan et al., 2024; Xie et al., 2025; Faghih et al., 2025), but ICVE moves evidence search, repair, mutation, and stopping into executable intent-level state machines. Semantic parsing and task-oriented dialogue use intent/slot structure; ICVE treats frames as substrates for checked execution.

7 Conclusion

ICVE reframes reliable stateful tool use as runtime-checked execution: the agent supplies intent, while a typed runtime owns risky transitions. Across covered ToolSandbox and AppWorld slices, this boundary reduces invalid calls, unsafe changes, and repair cost; broader leaderboard coverage remains next.

Limitations

The current strongest claim is intentionally narrow. The clean real-LLM evidence is on ToolSandbox no-distraction single-turn and insufficient-information suites plus targeted AppWorld transfer, complete local official AppWorld dev57, and local `test_normal.txt` file-level runs. These runs use AppWorld's packaged evaluator where applicable, but they are not a public AppWorld leaderboard submission. Unsupported `test_challenge` product-search purchase, many Gmail, and vacation-expense tasks remain blocked as unsupported no-action results rather than forced through similar-looking machines. Multi-turn cov-

failure type	ReAct manifestation	guardrail manifestation	ICVE mechanism	evidence position
Invalid call	Fragile JSON/API arguments and repeated failed tool calls.	Rejects malformed or ungrounded contracts, then asks the model to retry.	Schema-checked IntentFrame; deterministic handler emits validated tool arguments.	Tables 2, 3.
Missing evidence	Model guesses values or searches ad hoc before mutation.	Requires evidence fields but still relies on model-authored proof/action pairs.	Ledger-driven evidence acquisition and grounded slot checks before mutation.	Section 4.3; AppWorld dev57 and test-normal diagnostics.
Precondition uncertainty	Text loop repairs settings or payment state after failures.	Verifier detects failed preconditions but incurs rejection/repair loops.	Bounded runtime repair before a mutation, followed by postcondition stopping.	ToolSandbox repair columns and AppWorld Venmo card probes.
Insufficient information	ReAct may mutate despite missing user evidence.	Reduces unsafe calls, but retains nonzero unsafe/invalid termination and model cost.	Abstention frame or guarded final action with no model-owned mutation loop.	Table 3; Fig. 2.
Uncovered intent	Agent may try plausible tools for a similar-looking task.	Coverage is implicit in prompt/verifier behavior.	No registered machine means no high-risk mutation; static and executable coverage gaps are reported.	AppWorld coverage audit.

Table 8: Execution-boundary failure taxonomy. Covered high-risk failures are converted from model-owned mutation loops into checked transitions, deterministic repair, or safe abstention.

erage is promising but not yet a headline claim.

The explicit, partly hand-engineered registry improves auditability but limits open-domain coverage. Our affordance-induction prototype shows one conservative extension path: log unsupported intent frames, induce schemas and handlers only for signature-identifiable boolean setting APIs, execute candidates first in shadow mode, and promote them only after runtime invariants and counterexample tests pass. This improves coverage for a recognizable API family, but it is not open-ended program synthesis. Extending the idea to AppWorld purchase, Gmail, vacation-expense, or other multi-entity workflows remains future work; newly proposed machines must be checked before they are trusted to mutate state.

The runtime does not eliminate the need for a language model. It changes where the model is trusted. ICVE still needs the model for uncovered intents, natural-language clarification, and domains whose state machines have not been registered. The safety claim therefore applies to covered high-risk intents, not arbitrary tool execution.

Finally, hosted-model coverage remains narrow: the evidence spans Qwen2.5 scales, a small Phi-3-mini diagnostic, and hosted DeepSeek replications, but not a broad set of proprietary frontier models.

Ethical Considerations

Reliable stateful tool execution has positive applications in reducing accidental mutations, repeated tool calls, and unsafe guessing by deployed agents. The same runtime discipline could also make high-impact automated systems more capable, so the method should be paired with domain authorization, audit logs, and tool-level access controls. ICVE is not a substitute for policy enforcement: it checks covered state transitions against declared machine-readable rules, but cannot guarantee safety for tools or intents outside the registry.

References

BFCL. 2024. Berkeley function calling leaderboard. Berkeley Function Calling Leaderboard.

Hy Dang, Tianyi Liu, Zhuofeng Wu, Jingfeng Yang, Haoming Jiang, Tao Yang, Pei Chen, Zhengyang Wang, Helen Wang, Huasheng Li, Bing Yin, and Meng Jiang. 2025. Improving large language models function calling and interpretability via guided-structured templates. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 24426–24442.

Kazem Faghih, Wenxiao Wang, Yize Cheng, Siddhant Bharti, Gaurang Sriramanan, Sriram Balasubramanian, Parsa Hosseini, and Soheil Feizi. 2025. Tool preferences in agentic LLMs are unreliable. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20954–20969.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116.

Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, Zezhong Wang, Yuxian Wang, Wu Ning, Yutai Hou, Bin Wang, Chuhan Wu, Xinzhi Wang, Yong Liu, Yasheng Wang, and 8 others. 2025. ToolACE: Winning the points of LLM function calling. In *International Conference on Learning Representations*.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen

Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2024. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*.

Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. 2024. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. *arXiv preprint arXiv:2408.04682*.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large language model connected with massive APIs. In *NeurIPS*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris Maddison, and Tatsunori Hashimoto. 2024. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.

Weizhou Shen, Chenliang Li, Hongzhan Chen, Ming Yan, Xiaojun Quan, Hehong Chen, Ji Zhang, and Fei Huang. 2024. Small LLMs are weak tool learners: A multi-LLM agent. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 16658–16680.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076.

Hongru Wang, Wenyu Huang, Yufei Wang, Yuanhao Xi, Jianqiao Lu, Huan Zhang, Nan Hu, Zeming Liu, Jeff Z. Pan, and Kam-Fai Wong. 2025. Rethinking stateful tool use in multi-turn dialogues: Benchmarks and challenges. *arXiv preprint arXiv:2505.13328*.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*.

Yuejin Xie, Youliang Yuan, Wenxuan Wang, Fan Mo, Jianmin Guo, and Pinjia He. 2025. ToolSafety: A comprehensive dataset for enhancing safety in LLM-based agent tool invocations. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 14135–14156.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*.

A Claim-to-Evidence Matrix

Table 9 summarizes the main claims, the evidence that would make each claim falsifiable, and the boundary under which the claim should be read. The matrix is not a substitute for the main experiments; it is a compact audit trail for interpreting the results without treating covered-task success as open-domain tool-use safety.

Claim	What would weaken or falsify it	Evidence location	Boundary
Stateful failures are execution-boundary failures, not only reasoning failures.	ReAct or ReAct+guardrail would match ICVE's safety and cost while keeping the model-owned loop.	ToolSandbox insufficient-info: ReAct+guardrail reaches .750 success but still has .179 unsafe mutations, 1.036 invalid calls, 2.214 verifier rejections, and 6.786 LLM calls per task; ICVE reaches 1.00 success with zero unsafe, zero invalid, and zero agent LLM calls. The MiniStore diagnostic shows the same proof/retry cost pattern.	Covered stateful tool-use families, not arbitrary agents.
ICVE is more than a per-task script bank.	The registry would have near one machine per benchmark task with no cross-task reuse.	ToolSandbox uses 13 static machines. AppWorld has 125 registered machines; 55 used machines cover 168 `test_normal.txt` tasks, or 3.05 tasks per used machine. The artifact lists per-machine slots, compiler LOC, handler LOC, total LOC, and API namespaces.	Manual engineering remains part of the current method.
The AppWorld machines are not written from hidden answers.	Execution would require task answers, private database state, compiled solutions, or successful trajectories.	The runner loads only public instructions and accesses state through live `apis`; policies block task-file, ground-truth, compiled-solution, and oracle references. The static coverage audit compiles instructions without starting AppWorld, executing tools, inspecting databases, or loading ground truth.	This is an implementation audit, not a proof of independent third-party development.
Typed intent recognition alone does not explain the AppWorld gains.	Giving the model the same typed frame and asking it to write mutation code would close the gap.	On the 72-task AppWorld slice, Qwen typed-intent/code reaches 0/72 and DeepSeek typed-intent/code reaches 7/72, while ICVE runtime rows reach 72/72 with zero unsafe changes.	Targeted AppWorld slice and local evaluator evidence.
Coverage gaps become reported no-action outcomes rather than hidden unsafe mutation.	Unsupported or similar-looking tasks would be forced through machines and mutate state unsafely.	On local `test_normal.txt`, ICVE supports and succeeds on 168/168 tasks with zero unsafe state changes. A static public-instruction audit compiles only 138/417 `test_challenge` tasks, and the executable prefix still leaves 9/24 Amazon product-search purchase tasks unsupported under conservative Amazon machines, again with zero unsafe changes. The derived coverage roadmap records the required machine capabilities and validation gates before these buckets should be claimed as supported.	Lower success is expected outside registered coverage.
The dynamic path is a bounded coverage extension, not open-ended synthesis.	The inducer would fail on unseen regular affordances or accept unrelated/invalid APIs.	With no static machines, five unseen boolean-setting APIs are induced and executed correctly; an invalid `focus_mode` setter and an unrelated stock query are rejected. Promotion requires shadow-mode execution, invariant checks, and counterexample tests.	Regular boolean setting APIs only.
The pattern is not tied to one local model family.	The effect would disappear outside Qwen2.5-3B.	The paper reports Qwen2.5-0.5B and 7B-4bit scale checks, Phi-3-mini diagnostics, and hosted DeepSeek-chat / DeepSeek-reasoner ToolSandbox replications.	Not broad proprietary frontier-model coverage.

Table 9: Claim-to-evidence matrix. Each row states how the claim should be interpreted, where the supporting evidence is reported, and what boundary prevents overclaiming.

B Artifact and Reproducibility Map

The anonymous supplement is organized so that each reported result has a source-code entry point and a recorded summary directory. The main files are:

- The runtime and DSL live under `src/pctu_pilot/`; the key modules are the typed DSL, registry runtime, ToolSandbox binding, and AppWorld binding.
- Public runners live under `experiments/`; they cover ToolSandbox, AppWorld, dynamic setting-machine induction, and held-out boolean-affordance induction.
- Static AppWorld coverage results live in `results/appworld_static_coverage`; they audit public-instruction compile coverage for local `test_normal.txt` and `test_challenge.txt` without executing tools or loading ground truth, and include `coverage_roadmap.csv` for unsupported buckets.
- Review-strengthening summaries live in the dated `icve_review_strengthening` result directory; it contains `summary.json` and the per-machine development-cost table.
- The full artifact index is the `artifact_manifest_rave2.md` file under `paper/`; it maps paper tables to result directories and records the scope boundaries used in the paper.

The supplement builder and package verifier are both in `experiments/`. The verifier checks upload hashes, PDF section placement, supplement text hygiene, redaction of raw model-output preview fields, absence of raw trajectories, and freshness of the local transfer sheets. These checks do not prove the scientific claims by themselves; they verify that the submitted artifacts match the paper and that the reported evidence is traceable.